



Documentação Técnica Completa do Vest.IA

Sistema de Guarda-Roupa Digital

Equipe de Desenvolvimento — Grupo 2 / PIEC1

Reinaldo Sergio Dias da Silva

José Ednaldo dos Santos Filho

Luiz Eduardo Cordeiro Araujo

Everton Guilherme Marinho Silva

William Rodrigues de Freitas

1. Visão Geral do Sistema

O Vest.IA é uma aplicação web que permite ao usuário digitalizar e organizar seu guarda-roupa pessoal com o apoio de inteligência artificial. O sistema identifica automaticamente o tipo, a cor, a ocasião e o clima ideal de cada peça a partir de fotos enviadas pelo próprio usuário, e usa essas informações para sugerir combinações de looks adequadas ao dia a dia.

O grande diferencial do projeto está na integração entre três fontes de informação: o guarda-roupa cadastrado pelo usuário, os dados climáticos obtidos em tempo real pela internet e um modelo de visão computacional especializado em moda (Fashion-CLIP). Juntos, esses elementos permitem que o sistema sugira looks personalizados de forma automática, sem que o usuário precise configurar nada manualmente.

1.1. Problema que o Sistema Resolve

É comum que pessoas tenham diversas peças de roupa no armário e acabem usando sempre as mesmas, esquecendo das demais. O Vest.IA combate isso registrando quantas vezes cada peça foi usada e priorizando as menos frequentes nas sugestões. Além disso, elimina a dificuldade de montar looks no dia a dia ao oferecer combinações prontas com base no clima e na ocasião.

1.2. Principais Funcionalidades

- Cadastro de roupas com classificação automática por IA a partir de fotos
- Sugestão de looks completos com base no clima atual e na ocasião desejada
- Busca personalizada por texto livre (ex.: "quero ir à faculdade com frio")
- Controle de uso das peças com histórico e contagem de utilizações
- Gerenciamento de conta com troca de senha e foto de perfil
- Cadastro seguro com verificação por código enviado por e-mail

2. Arquitetura do Sistema

O Vest.IA segue a arquitetura MVC (Model-View-Controller), um padrão clássico que separa o sistema em três camadas bem definidas: dados, lógica e apresentação. O framework utilizado é o Flask (Python), e o banco de dados é o SQLite.

2.1. Estrutura de Arquivos

```
VEST.IA/
├── app.py                # Ponto de entrada da aplicação
├── routes.py             # Rotas, endpoints e lógica de controle
├── banco_dados/
│   ├── create_db.py     # Criação das tabelas do banco
│   └── database.py       # Funções de acesso e consulta ao banco
├── modulos/
│   ├── ia_classificador.py # Classificação de roupas com Fashion-CLIP
│   └── ia_sugestoes.py    # Lógica de sugestão e montagem de looks
├── utils/
│   ├── auth_utils.py    # Controle de autenticação (decorador)
│   ├── clima_utils.py   # Consulta de clima via API Open-Meteo
│   ├── email_utils.py   # Envio de e-mails via Brevo
│   └── imagem_utils.py  # Salvamento e conversão de imagens
├── templates/           # Páginas HTML (Jinja2)
├── static/              # Imagens, logos e uploads de roupas
├── tests/               # Testes automatizados
└── test_app.py          # Testes de integração e unitários
```

2.2. Tecnologias Utilizadas

Camada	Tecnologia	Para que serve
Framework Web	Python / Flask	Recebe as requisições do navegador e retorna as respostas
Banco de Dados	SQLite3	Armazena usuários, roupas, fotos e histórico de uso
Segurança de Senha	Werkzeug (PBKDF2)	Criptografa as senhas antes de salvar no banco
Interface Visual	HTML5 / Tailwind CSS	Páginas do sistema, responsivas para celular e computador
Motor de IA	PyTorch / HuggingFace	Infraestrutura para rodar o modelo de visão computacional
Modelo de Visão	Fashion-CLIP	Identifica tipo, cor, ocasião e clima ideal das roupas por foto

Clima	API Open-Meteo	Fornece a temperatura atual para ajustar as sugestões de look
E-mail	API Brevo	Envia os códigos de verificação para o e-mail do usuário

3. Descrição Detalhada dos Módulos

3.1. app.py — Inicializador da Aplicação

Este arquivo é o ponto de partida de todo o sistema. Ao ser executado, ele cria a aplicação Flask, define a chave de segurança das sessões (necessária para manter o usuário logado entre páginas), configura a pasta de destino das fotos enviadas e estabelece um limite máximo de 16 MB por upload. Antes de abrir o servidor, ele garante que o banco de dados já existe — criando-o automaticamente se for a primeira vez.

Porta padrão: 7860 | **Extensões de imagem aceitas:** PNG, JPG, JPEG, GIF, WEBP

3.2. routes.py — Rotas e Lógica de Controle

É o arquivo mais extenso do projeto. Centraliza todas as rotas da aplicação — tanto as páginas HTML quanto os endpoints de API consumidos pelo JavaScript do front-end. Está organizado em grupos funcionais:

Páginas Principais

Rota	Função
/ e /login	Tela de login. Redireciona para o dashboard se já estiver logado.
/register	Tela de cadastro de nova conta.
/dashboard	Painel inicial com total de roupas e peças menos utilizadas.
/cadastrar	Tela para adicionar novas roupas com foto.
/sugestoes	Tela com os looks sugeridos pelo sistema.
/minhas_roupas	Lista todas as peças do guarda-roupa do usuário.
/historico_page	Histórico cronológico de roupas utilizadas.
/perfil	Página de gerenciamento de dados da conta.

Autenticação e Segurança

O decorador `@login_obrigatorio` é aplicado a todas as rotas que exigem sessão ativa. Se o usuário tentar acessar uma página protegida sem estar logado, é redirecionado automaticamente para a tela de login.

O cadastro é realizado em duas etapas: o sistema envia um código numérico de 6 dígitos para o e-mail informado e só efetiva o registro após a confirmação desse código. O mesmo processo é usado para recuperação de senha e para alteração de e-mail.

Guarda-Roupa

A rota `/cadastrar_via_imagem` recebe as fotos selecionadas pelo usuário e aciona o módulo de IA para classificar cada peça. O resultado (tipo, cor, ocasião, clima) é retornado ao navegador para que o usuário possa revisar antes de confirmar o salvamento. A rota `/salvar_final` então persiste os dados no banco.

Outras rotas gerenciam o ciclo de vida das peças: `/listar` retorna o catálogo completo, `/editar/<id>` atualiza os atributos de uma peça, `/excluir/<id>` remove a peça e seus arquivos físicos do servidor, e `/usar/<id>` registra uma utilização no histórico.

APIs de Sugestão

Endpoint	O que faz
<code>/api/sugestoes_combinacoes</code>	Retorna três looks prontos (para calor, frio e meia-estação) com base nas roupas cadastradas e no clima atual.
<code>/api/busca_personalizada</code>	Interpreta um texto livre do usuário e filtra as peças mais adequadas ao contexto descrito.
<code>/api/stylist_look</code>	Monta um look completo (superior + inferior + calçado) baseado na mensagem do usuário, com resposta em linguagem natural.
<code>/sugerir_combinacoes/<id></code>	Sugere peças compatíveis com uma peça específica selecionada pelo usuário (mesma ocasião e clima, tipo complementar).

4. Banco de Dados

O banco de dados utilizado é o SQLite, armazenado localmente no arquivo `banco_dados/vest.ia.db`. A conexão é configurada com Write-Ahead Logging (WAL), que melhora o desempenho em situações de acesso simultâneo, e com chaves estrangeiras ativadas, garantindo que não seja possível cadastrar uma roupa vinculada a um usuário inexistente, por exemplo.

4.1. Tabelas

usuarios

Armazena os dados de cada pessoa cadastrada no sistema.

Campo	Tipo	Descrição
<code>id</code>	INTEGER	Identificador único gerado automaticamente
<code>nome</code>	TEXT	Nome do usuário
<code>email</code>	TEXT	E-mail único (usado no login)
<code>senha</code>	TEXT	Senha criptografada com hash PBKDF2
<code>foto_url</code>	TEXT	Caminho da foto de perfil (opcional)
<code>data_cadastro</code>	DATETIME	Data e hora do cadastro

roupas

Armazena cada peça cadastrada, sempre vinculada a um usuário.

Campo	Tipo	Descrição
-------	------	-----------

id	INTEGER	Identificador único
usuario_id	INTEGER	Chave estrangeira para a tabela usuarios
nome	TEXT	Nome da peça (gerado automaticamente ou editado pelo usuário)
tipo	TEXT	Ex.: Camiseta, Calça, Casaco, Calçado...
cor	TEXT	Ex.: Azul, Preto, Listrado, Colorido...
ocasio	TEXT	Ex.: Casual, Trabalho, Festa, Formal, Esporte
clima_ideal	TEXT	Calor, Frio ou Meia-estação
vezes_usada	INTEGER	Contador de utilizações (começa em zero)
data_cadastro	DATETIME	Data e hora do registro

fotos_roupas

Uma roupa pode ter várias fotos. Esta tabela armazena o caminho de cada imagem associada a uma peça.

Campo	Tipo	Descrição
id	INTEGER	Identificador único
roupa_id	INTEGER	Chave estrangeira para a tabela roupas
caminho	TEXT	Caminho relativo da imagem no servidor (ex.: /static/uploads/Calça/foto.jpg)

historico

Registra cada vez que o usuário marca uma peça como "utilizada hoje".

Campo	Tipo	Descrição
id	INTEGER	Identificador único
roupa_id	INTEGER	Chave estrangeira para a tabela roupas
data_uso	TEXT	Data e hora em que a peça foi marcada como usada

5. Módulos de Inteligência Artificial

5.1. ia_classificador.py — Classificação de Roupas por Foto

Este módulo é responsável por analisar as imagens enviadas pelo usuário e determinar quatro atributos de cada peça: tipo, cor, ocasião e clima ideal.

Para isso, utiliza o modelo Fashion-CLIP, uma versão especializada em moda do modelo CLIP (da OpenAI), ajustada pelo pesquisador Patrick John Cyh. O modelo compara a imagem enviada com descrições textuais de cada categoria e calcula qual descrição é mais compatível com a foto.

Como funciona a classificação

Para cada atributo, existe um conjunto de candidatos com descrições em português. Por exemplo, para o tipo:

- **Camisa:** "uma camisa social de botão de manga longa ou curta"
- **Camiseta:** "uma camiseta casual básica de algodão manga curta t-shirt"
- **Casaco:** "um casaco grosso, jaqueta, moletom, blazer ou casaco de frio"
- ... e assim por diante para todos os 13 tipos suportados

O modelo retorna o candidato com maior semelhança visual-textual. Quando o usuário envia mais de uma foto da mesma peça, o sistema realiza uma votação: o atributo mais votado entre todas as fotos é o escolhido.

Atributos Classificados

Atributo	Opções Disponíveis	Observação
Tipo	Camisa, Camiseta, Camisa de Time, Blusa, Casaco, Calça, Bermuda, Saia, Vestido, Acessório, Calçado	13 categorias com descrições detalhadas
Cor	Preto, Branco, Cinza, Azul, Vermelho, Verde, Amarelo, Rosa, Marrom, Bege, Listrado, Xadrez, Colorido	Inclui padrões além de cores sólidas
Ocasão	Casual, Formal, Trabalho, Festa, Esporte	5 contextos de uso
Clima Ideal	Calor, Frio, Meia-estação	Baseado na leveza/espessura visual da peça

O modelo é carregado na memória uma única vez quando o servidor inicia (em cache local na pasta `instance/fashion-clip/`). Se o modelo não estiver presente, ele é baixado automaticamente do HuggingFace. A execução ocorre em GPU (CUDA) se disponível, ou em CPU caso contrário.

5.2. `ia_sugestoes.py` — Motor de Sugestão de Looks

Este módulo contém toda a lógica de recomendação de looks. Não usa aprendizado de máquina — funciona com regras e pontuações definidas manualmente, o que o torna rápido e previsível.

`detectar_intencao()`

Analisa o texto livre digitado pelo usuário (ex.: "quero algo para ir ao trabalho com frio") e extrai quatro informações: ocasião desejada, clima, cor preferida e tipo de peça a excluir. Isso é feito por comparação de palavras-chave mapeadas em dicionários.

`_score_peca()`

Calcula uma pontuação de adequação para cada peça do guarda-roupa com base na intenção detectada:

- +4 pontos se a ocasião da peça coincidir com a intenção
- +3 pontos se o clima ideal da peça coincidir
- +2 pontos se a cor coincidir
- -0,1 ponto por cada vez que a peça já foi usada (favorece variedade)

`montar_look()`

Seleciona as melhores peças para compor um look completo. A estratégia é montar um conjunto com uma peça superior (camisa/camiseta/blusa), uma inferior (calça/bermuda/saia) e um calçado. Se o guarda-roupa tiver apenas vestidos, monta um look com vestido + calçado. As peças são escolhidas por pontuação e peças já mostradas anteriormente são excluídas para evitar repetição.

`gerar_look_por_clima()`

Monta um look aleatório para um clima específico (calor, frio ou meia-estação). É usado nos cards automáticos da tela de sugestões, onde o sistema exibe três looks prontos sem que o usuário precise digitar nada.

`gerar_texto_look()`

Gera o texto de resposta que o sistema apresenta ao usuário após montar um look. Inclui um título, uma descrição com as peças escolhidas, a paleta de cores e uma dica de estilo para a ocasião.

6. Módulos Utilitários (utils/)

6.1. `auth_utils.py` — Controle de Acesso

Contém apenas o decorador `@login_obrigatorio`. Quando aplicado a uma rota, verifica se há um `usuario_id` válido na sessão atual. Se não houver, redireciona o usuário para a página de login com uma mensagem de aviso.

6.2. `clima_utils.py` — Consulta de Clima

Consulta a API pública Open-Meteo para obter a temperatura atual em graus Celsius. A localização padrão configurada é Belo Jardim, PE (latitude -8.33, longitude -36.42). A temperatura é então classificada em três faixas:

- **Calor:** temperatura igual ou acima de 25°C
- **Frio:** temperatura igual ou abaixo de 19°C
- **Meia-estação:** entre 20°C e 24°C

Se a consulta falhar por qualquer motivo (falta de internet, timeout, erro da API), o módulo retorna meia-estação como valor padrão, evitando que a página de sugestões quebre.

6.3. `email_utils.py` — Envio de E-mails

Utiliza a API da Brevo (antiga Sendinblue) para enviar os códigos de verificação por e-mail. O remetente configurado é `vestia.noreply@gmail.com`. O e-mail enviado contém o código em destaque, com um layout HTML simples. A chave da API é lida a partir de variável de ambiente (`BREVO_API_KEY`), nunca armazenada no código.

6.4. `imagem_utils.py` — Processamento de Imagens

Fornece duas funções para salvar imagens no servidor. A primeira (`salvar_imagem_base64`) recebe uma imagem em formato base64, converte para JPEG usando a biblioteca `Pillow` e salva na pasta correspondente ao tipo da peça. A segunda (`salvar_foto_perfil`) faz o mesmo processo para a foto de perfil do usuário, salvando na pasta `static/uploads/perfil/`. Todos os arquivos recebem um nome único gerado aleatoriamente (UUID).

7. Fluxos Principais do Sistema

7.1. Cadastro de Conta com Verificação por E-mail

- Usuário preenche nome, e-mail e senha na tela de cadastro
- Sistema verifica se o e-mail já está em uso
- Um código de 6 dígitos é gerado e enviado para o e-mail informado
- O registro fica pendente na sessão até a confirmação do código
- Após confirmar o código correto, a conta é criada e o usuário é logado automaticamente

7.2. Cadastro de Roupas com IA

- Usuário acessa a tela "Cadastrar Roupas" e seleciona uma ou mais fotos
- As fotos são enviadas ao servidor e processadas pelo Fashion-CLIP
- O sistema retorna tipo, cor, ocasião e clima detectados para revisão
- Usuário pode confirmar ou ajustar os dados antes de salvar
- Ao confirmar, as fotos são salvas na pasta correspondente ao tipo e os dados são gravados no banco

7.3. Sugestão de Look pelo Chat

- Usuário digita uma mensagem descrevendo o que precisa (ex.: "preciso ir a uma festa hoje à noite")
- O sistema detecta a intenção: ocasião = Festa, clima = obtido da API em tempo real
- As peças do guarda-roupa são pontuadas e as melhores de cada categoria são selecionadas
- Um look completo (superior + inferior + calçado) é montado e apresentado com texto explicativo
- O usuário pode pedir um novo look, que exclui as peças já mostradas

7.4. Recuperação de Senha

- Usuário informa o e-mail cadastrado
- Sistema verifica se o e-mail existe e envia um código de 6 dígitos
- Usuário informa o código recebido
- Após validação, o sistema permite cadastrar uma nova senha

8. Segurança

8.1. Senhas

As senhas nunca são armazenadas em texto puro. Ao cadastrar ou alterar a senha, o sistema aplica o algoritmo PBKDF2 com salting automático via a biblioteca Werkzeug. Na autenticação, a senha digitada é comparada ao hash armazenado — nunca ao texto original.

8.2. Proteção de Rotas

Todas as rotas que exigem login usam o decorador `@login_obrigatorio`. O identificador do usuário é armazenado na sessão Flask, que é assinada criptograficamente pela chave secreta definida em `FLASK_SECRET_KEY` (variável de ambiente).

8.3. Isolamento de Dados

Todas as operações de banco de dados (buscar, editar, excluir) incluem o `usuario_id` como condição obrigatória. Isso impede que um usuário acesse ou modifique dados de outro usuário, mesmo que descubra o ID de uma peça alheia.

8.4. Tamanho Máximo de Upload

O sistema rejeita arquivos maiores que 16 MB, evitando abusos de armazenamento no servidor.

8.5. Variáveis de Ambiente

Chaves sensíveis (`FLASK_SECRET_KEY`, `BREVO_API_KEY`) são carregadas de um arquivo `.env` pelo pacote `python-dotenv`, nunca escritas diretamente no código.

9. Páginas da Interface (templates/)

Arquivo	Descrição
<code>layout.html</code>	Template base herdado por todas as outras páginas. Contém a barra lateral de navegação, scripts comuns e área de notificações (toasts).
<code>login.html</code>	Tela de entrada com formulário de e-mail e senha.
<code>register.html</code>	Cadastro em duas etapas: dados pessoais e confirmação por código de e-mail.
<code>dashboard.html</code>	Painel inicial com o total de roupas cadastradas e as peças menos utilizadas.
<code>cadastrar_roupa.html</code>	Upload de fotos com pré-visualização, exibição dos dados classificados pela IA e edição antes de salvar.
<code>minhas_roupas.html</code>	Catálogo completo do guarda-roupa com filtros por tipo, cor e ocasião. Permite editar, excluir e marcar peças como usadas.
<code>sugestoes.html</code>	Tela principal de recomendações: cards automáticos por clima e chat interativo com o stylist virtual.
<code>historico.html</code>	Lista cronológica de todas as roupas marcadas como usadas.
<code>perfil.html</code>	Página de conta: troca de foto de perfil e alteração de senha.
<code>alterar_dados.html</code>	Tela para atualização de nome e e-mail com verificação de segurança.
<code>esquecisenha.html</code>	Fluxo completo de recuperação de senha por e-mail.

10. Configuração e Variáveis de Ambiente

O arquivo `.env` está na raiz do projeto com as seguintes variáveis:

```
FLASK_SECRET_KEY=chave_secreta_aqui
BREVO_API_KEY=chave_da_api_brevo_aqui
```

FLASK_SECRET_KEY: Chave usada para assinar os cookies de sessão. Pode ser qualquer string longa e aleatória.

BREVO_API_KEY: Chave obtida no painel da plataforma Brevo, necessária para o envio de e-mails.

11. Testes Automatizados

Os testes do projeto estão localizados na pasta `tests/`, seguindo a convenção de boas práticas para projetos Python. O arquivo principal é `tests/test_app.py`, executado com o framework `pytest`.

11.1. Configuração do Ambiente de Testes

Um fixture chamado `cliente` configura o Flask em modo de testes (`TESTING = True`) e disponibiliza um cliente HTTP simulado para todas as funções de teste. Isso permite enviar requisições ao servidor sem precisar iniciá-lo de verdade.

```
# Executar todos os testes
pytest tests/
# Com detalhes: pytest tests/ -v
```

11.2. Testes de Integração e Segurança

Verificam o comportamento real das rotas HTTP:

Teste	Rota	O que verifica
<code>test_pagina_login_status</code>	GET <code>/login</code>	Retorna HTTP 200 e renderiza o formulário de login.
<code>test_seguranca_dashboard_sem_login</code>	GET <code>/dashboard</code>	Usuário sem sessão deve receber redirecionamento HTTP 302.

11.3. Testes Unitários da IA

Verificam a lógica interna do módulo `ia_sugestoes` sem depender do servidor. O teste `test_regras_de_ocasiao_ia` valida o dicionário `MAPA_OCASIAO`: casamento retorna Formal, academia retorna Esporte, shopping retorna Casual, reuniao retorna Trabalho, e palavras desconhecidas retornam None (caso negativo).

11.4. Localização

```
tests/
  test_app.py      # Todos os testes do projeto
```

12. Considerações Finais

O Vest.IA é um projeto acadêmico completo que combina backend web, banco de dados relacional, inteligência artificial local e integração com APIs externas. A arquitetura modular facilita a manutenção e a evolução do sistema — cada responsabilidade está isolada em seu próprio arquivo, tornando simples a localização e correção de problemas.

Como evolução futura, o projeto poderia incorporar um sistema de recomendação baseado no histórico de uso do próprio usuário, a possibilidade de compartilhar looks com amigos, e suporte a múltiplos idiomas.